

LC827: Making A Large Island ★

Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 22, 2026

Sprinklr

Jio

Snowflake

We can change any one 0 to 1 and we need to return the max possible island area.

Brute Force Approach

for every 0 → flip it → run DFS → restore it → update answer

Time Complexity: $TC = O((n \times m)^2)$

Optimal Approach

Give every island a unique id and store each of their area in a hashmap.

Example Grid

1	1	0
0	1	0
1	0	1



After ID Assignment

2	2	0
0	2	0
3	0	4

S1 DFS every island, give them unique ID, and store their area in HashMap.

S2 Visit every zero, collect neighbouring IDs: $1 + \text{sum}(\text{unique IDs})$.

Time Complexity: $TC = O(n \times m) + O(n \times m) = O(2 \times n \times m)$

C++ Implementation

```

class Solution {
public:
    int largestIsland(vector<vector<int>>& grid) {

        int n = grid.size();
        int m = grid[0].size();
        // step 1 , sabhi islands pe dfs krke unka area store kr lenge idwise

        vector<pair<int,int>> dir = {{1,0}, {0,1}, {-1,0}, {0,-1}};
        unordered_map<int,int> mp;

        int id = 2; // 0 and 1 are reserved for land and water

        queue<pair<int,int>> q;
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(grid[i][j] == 1) {
                    grid[i][j] = id;
                    int curr = 1; // current island size
                    q.push({i, j});

                    while(!q.empty()) {
                        int r = q.front().first;
                        int c = q.front().second;
                        q.pop();

                        for(auto it : dir) {
                            int nr = r + it.first;
                            int nc = c + it.second;

                            if(nr >= 0 && nc >= 0 && nr < n && nc < m) {
                                if(grid[nr][nc] == 1) {
                                    grid[nr][nc] = id;
                                    curr++;
                                    q.push({nr, nc});
                                }
                            }
                        }
                    }

                    mp[id] = curr;
                    id++;
                }
            }
        }

        int ans = 0;
        bool foundzero = false;
        for(int i = 0; i < n; i++) {

```

```

        for(int j = 0; j < m; j++) {
            if(grid[i][j] == 0) {
                foundzero = true;
                int currans = 1;
                unordered_set<int> st;
                for(auto it : dir) { // har direction me jao aur dekho pados me
                                    // 0 can act as a pivot to join them
                    int nr = i + it.first;
                    int nc = j + it.second;

                    if(nr >= 0 && nc >= 0 && nr < n && nc < m) {
                        if(grid[nr][nc] >= 2) {
                            st.insert(grid[nr][nc]);
                        }
                    }
                }

                for(auto it : st) {
                    currans += mp[it];
                }

                ans = max(ans, currans);
            }
        }

        if(foundzero)
            return ans;
        else
            return n * m; // because agar 0 nahi mila matlab saare hi 1 hain
    }
};

```